

AI Document Assistant - Technical Design Corpus

Benchmark corpus for evaluating a single-document RAG application. This document describes the architecture, implementation choices, usage tracking, storage model, limitations, and intentional non-goals of the AI Document Assistant project.

The document is intentionally structured for retrieval evaluation. Each section contains concrete facts, numbers, constraints, and trade-offs that can be used to create ground-truth questions for Recall@5 and Recall@10 benchmarks.

1. Product Goal

AI Document Assistant is a single-document RAG application. A user uploads one PDF, asks questions about that document, and receives streaming answers with citations to retrieved source chunks.

The primary goal is not to build a general knowledge base. The project focuses on practical AI engineering signals: real token-based chunking, swappable vector storage, pgvector persistence, streaming UI, cited answers, and measured cost and latency.

The application is designed as a portfolio project for demonstrating end-to-end AI application development with Next.js App Router and TypeScript strict mode.

1.1 Main user flow

The main flow is: upload PDF, extract text, split into chunks, embed chunks, store chunks, retrieve relevant chunks for a user question, generate a grounded answer, stream it to the UI, and show source cards.

A new upload replaces the active document. The application does not currently support asking questions across multiple documents.

2. Ingestion Pipeline

PDF ingestion is implemented as a Server Action. The upload path validates that the uploaded file is a PDF, extracts text with unpdf, computes document metadata, chunks the extracted text, embeds every chunk, and replaces the active document store.

The pipeline is intentionally synchronous for the portfolio scope. There are no background jobs, queues, webhooks, or separate workers.

2.1 PDF extraction

Text extraction uses unpdf. If the extracted text is empty, the UI returns an error indicating that the PDF may be scanned or image-only. OCR is not implemented.

2.2 Metadata

The document metadata includes a stable document id, filename, page count, character count, and createdAt timestamp. The UI shows product-facing metadata such as pages, tokens, and processed status. Internal details such as embedding dimensions are not shown to the user.

2.3 Document replacement

The project has single-active-document semantics. Replacing the document clears the previous active document and starts a new document session. The chat is also cleared after a successful replacement so old messages do not reference stale sources.

3. Chunking Strategy

Chunking is token-based, not character-based. The implementation uses gpt-tokenizer so chunk boundaries are based on the same unit family that LLMs consume.

The default chunk size is 500 tokens with 50 tokens of overlap. The overlap reduces the chance that a relevant sentence is split across chunk boundaries and lost during retrieval.

3.1 Why token-based chunking

A character heuristic such as four characters per token is too imprecise for retrieval. Token-based chunking gives more predictable prompt sizes and makes cost estimation more meaningful.

3.2 Chunk metadata

Each stored chunk includes text, optional token count, and metadata. Metadata includes documentId, filename, and chunk index. Page numbers and section headings are not currently propagated end to end, so source cards show filename and chunk number rather than fake page labels.

4. Embeddings

The embedding model is text-embedding-3-small. It returns 1536-dimensional vectors. The project uses batch embedding for upload so a document with many chunks does not produce one OpenAI request per chunk.

Embedding generation is separated from storage. The vector store receives ready embeddings and does not know about OpenAI or any provider-specific API.

4.1 Batch behavior

The batch size is 100 inputs per request. This reduces round trips, makes uploads more rate-limit friendly, and preserves the order of returned embeddings across batches.

4.2 Cost model

Embedding cost is estimated using token counts from gpt-tokenizer and the text-embedding-3-small price configured in the project. At the current pricing assumption, text-embedding-3-small costs 0.02 dollars per one million input tokens.

5. Storage Architecture

Storage is implemented behind a VectorStore interface. The interface allows the same retrieval logic to use either an in-memory store or a pgvector-backed store without changing the application pipeline.

The main implementations are MemoryStore and PgVectorStore. MemoryStore is useful for tests and local experimentation. PgVectorStore stores chunks in Supabase Postgres with the pgvector extension enabled.

5.1 VectorStore contract

VectorStore.search receives a ready embedding and topK. It does not accept raw text and does not generate embeddings internally. This keeps provider code out of the storage layer.

5.2 DocumentStore contract

Document lifecycle is handled by a separate DocumentStore abstraction. The active document replacement flow is backend-agnostic. In the Postgres path, replacing the active document deletes prior document rows and relies on foreign key cascade to remove associated chunks.

5.3 Supabase and pgvector

The Postgres schema has a documents table and a document_chunks table. document_chunks stores text, chunk index, filename, token count, optional page, and a 1536-dimensional vector embedding. The vector index uses cosine distance.

6. Retrieval

Retrieval uses dense vector similarity. A user question is embedded with the same embedding model, then retrieveChunks searches the vector store and returns the top matching chunks.

The current chat route uses TOP_K = 3 for answer generation. The benchmark harness can use k = 5 and k = 10 to compute Recall@5 and Recall@10.

6.1 Retrieval boundaries

The retrieveChunks function orchestrates query embedding and store search. The store remains embedding-only. This separation is a key design decision because it allows MemoryStore and PgVectorStore to share the same retrieval API.

6.2 Known retrieval limitations

The current version does not implement reranking, hybrid keyword plus semantic search, BM25, MMR, query rewriting, or multi-vector indexing. These are listed as future improvements rather than partially implemented features.

7. Answer Generation

Answer generation uses retrieved chunks as context. The chat route builds a prompt that instructs the model to answer only from the provided passages and cite supporting passages with bracketed citation numbers such as [1] and [2].

If the answer is not present in the retrieved context, the model is instructed to say: I could not find that in the document.

7.1 Streaming

The application streams answers using the Vercel AI SDK. The server route uses streamText and createUIMessageStreamResponse. Sources are sent as data parts after generation finishes, so source cards appear below the final answer.

7.2 Source cards

Source cards show the filename, chunk number, and a short preview of the retrieved chunk. Long sources can be expanded inline. The UI intentionally does not show similarity score, fake page numbers, or internal retrieval metadata.

8. Chat State

StreamingQuestionForm is the primary chat UI. The older QuestionForm flow has been removed from the product direction. Chat state is held in a shared Chat instance provided by ChatProvider, so the conversation survives client-side navigation between Upload and Assistant pages.

Hard refresh persistence is not implemented. The chat is cleared when a successful document replacement occurs. This prevents ghost citations and silent context switches between documents.

8.1 Why clear chat on replacement

Because the app supports one active document, keeping old chat messages after a replacement would imply multi-document continuity. Old citations would point to chunks that no longer belong to the active document. Clearing the chat keeps the UI consistent with the storage model.

9. Usage and Cost Tracking

Usage tracking records token and estimated cost information for embeddings and chat generation. The tracker is in-memory and intended for portfolio-level observability rather than billing.

The UI shows session usage in a compact sidebar card. The value resets when the server process restarts.

9.1 Pricing assumptions

The project tracks text-embedding-3-small input cost, gpt-4o-mini input cost, and gpt-4o-mini output cost. The configured assumptions are: text-embedding-3-small at 0.02 dollars per one million input tokens, gpt-4o-mini input at 0.15 dollars per one million tokens, and gpt-4o-mini output at 0.60 dollars per one million tokens.

9.2 Why estimated cost

Cost is presented as estimated because provider metadata can vary by API and streaming mode. Embedding tokens can be counted locally, while generation usage may combine provider metadata and fallback estimates depending on the route.

10. UI and Theme

The UI has two main sections: Upload and Assistant. Desktop uses a persistent left sidebar. Mobile and tablet use a top bar with an overlay drawer.

The application supports light, dark, and system themes through next-themes. The visual direction uses soft neutral surfaces, subtle borders, and violet accents. The UI is intentionally minimal and avoids dashboard charts that do not reflect real product data.

10.1 Upload UI

The Upload page includes a drag-and-drop area, click-to-upload support, a current document card, and helper copy explaining that uploading a new document replaces the current document and starts a new conversation.

10.2 Assistant UI

The Assistant page is conversation-first. It shows compact document context, user and assistant message bubbles, source cards attached to assistant answers, and a modern chat composer.

11. Evaluation and Benchmarks

The final benchmark uses a fixed corpus and a manually annotated question set. Questions map to expected chunk indices. The benchmark reports Recall@5, Recall@10, average retrieval latency, average end-to-end latency, average tokens per query, and average cost per query.

The project intentionally avoids LLM-as-judge evaluation. Deterministic retrieval metrics are cheaper, more reproducible, and easier to explain in a portfolio review.

11.1 Recall definition

Recall@k measures how many expected chunks appear in the top k retrieved chunks. The metric is averaged across the benchmark question set.

11.2 Benchmark scope

The benchmark is run on one PDF and 10 to 15 annotated questions. It can compare MemoryStore and PgVectorStore using the same ingestion and retrieval pipeline.

12. Non-goals

The project intentionally excludes authentication, multi-user accounts, multi-document chat, billing, background job queues, agent workflows, and production infrastructure hardening. These choices keep the project focused on RAG architecture, retrieval quality, and measurement.

The project also does not include OCR for scanned PDFs, advanced chunk layout analysis, reranking, hybrid search, or page-aware citation rendering. These are valid future improvements but not required for the current portfolio scope.

13. Future Work

Potential follow-up work includes page-aware chunk metadata, hybrid retrieval, reranking, per-document chat histories, persistent usage events, and internationalization with English and Ukrainian UI copy.

Another planned improvement is i18n using next-intl. The intended scope is English and Ukrainian UI strings, language switcher, and assistant answers that match the selected or detected language.

14. Summary

AI Document Assistant demonstrates a complete single-document RAG flow: ingest PDF content, chunk by tokens, embed with OpenAI, store vectors behind an interface, retrieve relevant chunks, stream grounded answers, show citations, and measure cost and latency.

The strongest engineering choices are the storage abstraction, provider-independent retrieval boundary, pgvector persistence, benchmarkable retrieval metrics, and explicit cost tracking.

Appendix A. Key Configuration

Setting	Value	Purpose
Chunk size	500 tokens	Controls retrieval granularity
Chunk overlap	50 tokens	Preserves context at boundaries
Embedding model	text-embedding-3-small	Low-cost 1536-dimensional embeddings
Chat model	gpt-4o-mini	Low-cost answer generation
Generation topK	3	Context passages used for answers
Benchmark k	5 and 10	Recall@5 and Recall@10 reporting
Embedding batch size	100 inputs	Reduces OpenAI round trips
Storage mode	Memory or pgvector	Selected by environment

Appendix B. Example Benchmark Questions

These are example question styles for annotation. The final benchmark questions should be created after extracting and previewing the real chunks from this PDF.

- What is the main product goal of AI Document Assistant?
- Which library is used for PDF extraction?
- What chunk size and overlap are used?
- Which embedding model is used and how many dimensions does it return?
- Why does the vector store receive embeddings instead of raw text?
- What does replacing the active document do to the chat?
- Which usage metrics are tracked in the UI?
- What are the main non-goals of the project?